



ROADMAP DE DÉVELOPPEMENT

Zümm

Planification Scrum & pipeline DevOps

Projet	Application de gestion apicole (mini-projet Java)
Type	Roadmap de développement (Scrum + DevOps)
Version	1.0
Date	16 juillet 2026
Référence	Cahier des charges Zümm v1.0 (13 juillet 2026)
Cadence	1 sprint de cadrage + 8 sprints de 14 jours — 14/07/2026 → 16/11/2026

10 epics — 40 user stories — 304 story points

Document conforme au plan : Méthodologie / Product Backlog / Sprints /
Risques & charge / Pipeline DevOps / Plan de releases / Monitoring & observabilité

Table des matières

Historique des versions	3
1 Méthodologie : Scrum + DevOps	4
1.1 Objet du document	4
1.2 Cadre Scrum	4
1.2.1 Rôles	4
1.2.2 Cérémonies	4
1.2.3 Definition of Ready (DoR)	5
1.2.4 Definition of Done (DoD)	5
1.3 Principes DevOps	5
2 Product Backlog	6
2.1 Vue d'ensemble des epics	6
2.2 EPIC-001 — Gestion des entités métier (CRUD)	6
2.3 EPIC-002 — Planification et suivi des visites	7
2.4 EPIC-003 — Tableaux de bord et visualisation	7
2.5 EPIC-004 — Indicateurs et capteurs (préparation)	8
2.6 EPIC-005 — Authentification et sécurité	8
2.7 EPIC-006 — Internationalisation et configuration	8
2.8 EPIC-007 — Service web API tierce	8
2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)	9
2.10 EPIC-009 — Intelligence artificielle (préparation)	9
2.11 EPIC-010 — Tests, qualité et documentation	9
3 Planification des sprints	10
3.1 Calendrier général	10
3.2 Synthèse des sprints	11
3.3 Recommandations d'équilibrage et de pilotage	11
3.3.1 Arbitrage retenu	12
3.3.2 Ordre des dépendances	13
3.3.3 Pilotage	13
3.3.4 Recommandations techniques transversales	13
3.4 Sprint 1 — Fondation : CRUD & configuration	14
3.4.1 Plan d'exécution	14
3.5 Sprint 2 — Ruche, i18n & socle qualité	15

3.5.1	Plan d'exécution	15
3.6	Sprint 3 — Visites & rapports	16
3.6.1	Plan d'exécution	17
3.7	Sprint 4 — Hors-ligne, authentification & RBAC	17
3.7.1	Plan d'exécution	18
3.8	Sprint 5 — Tableaux de bord & pilotage	19
3.8.1	Plan d'exécution	19
3.9	Sprint 6 — Synthèse, capteurs & API	20
3.9.1	Plan d'exécution	20
3.10	Sprint 7 — Fonctions avancées & détection d'anomalie	21
3.10.1	Plan d'exécution	21
3.11	Sprint 8 — Qualité & livraison	22
3.11.1	Plan d'exécution	22
3.12	Suivi en cours de sprint	22
4	Gestion des risques et de la charge	24
4.1	Registre des risques	24
4.2	Charge estimée par sprint	25
4.3	Suivi de la vitesse	26
5	Pipeline DevOps (CI/CD)	27
5.1	Environnements	27
5.2	Vue d'ensemble du pipeline	27
5.3	Détail des étapes	28
5.4	Infrastructure as Code	28
5.5	Mise en place progressive	28
5.6	Stratégie de branches Git	29
6	Plan de releases	30
6.1	Synthèse	30
6.2	v0.1.0 — MVP : Fondation	30
6.3	v0.2.0 — Visites & collaboration	30
6.4	v0.3.0 — Analytics & dashboards	31
6.5	v1.0.0 — Release Candidate	31
7	Monitoring & observabilité	32
7.1	Dashboards Grafana	32
7.1.1	Zümm — Vue d'ensemble (métier)	32
7.1.2	Zümm — Performance (technique)	32
7.1.3	Zümm — Sécurité	32
7.2	Règles d'alerte	33
7.3	Gestion des logs	33

Historique des versions

Version	Date	Auteur	Modifications
1.0	16 juillet 2026	Équipe projet	Consolidation LaTeX du dossier <code>zumm_scrum_devops_roadmap</code> : backlog, sprints, pipeline CI/CD, releases et monitoring. Correction des totaux de points des sprints 5, 7 et 8.

CHAPITRE 1

Méthodologie : Scrum + DevOps

1.1 Objet du document

Ce document constitue la **roadmap de développement** du projet Zümm. Il traduit le cahier des charges (v1.0, 13 juillet 2026) en un plan d'exécution opérationnel : un *product backlog* priorisé, une planification en **8 sprints** de deux semaines avec plans d'exécution détaillés, un registre des risques coté, une chaîne d'intégration et de déploiement continu (CI/CD), un plan de releases sémantiques et un dispositif de monitoring.

Chiffres clés. 10 epics — 40 user stories — 304 story points — 1 sprint de cadrage + 8 sprints de 14 jours — 4 releases (v0.1.0 → v1.0.0) — période : 14 juillet → 16 novembre 2026.

1.2 Cadre Scrum

1.2.1 Rôles

Rôle	Responsabilité
Product Owner	Priorise le backlog, valide les critères d'acceptation, accepte les livrables en sprint review.
Scrum Master	Garant du cadre, lève les obstacles, anime les cérémonies.
Équipe de développement	Conçoit, développe, teste et livre l'incrément de chaque sprint.

1.2.2 Cérémonies

Chaque sprint suit le même rythme de cérémonies :

Cérémonie	Cadence / durée	Objectif
Sprint Planning	Jour 1, 09h00–13h00 (4 h)	Sélection des stories et engagement de l'équipe.
Daily Scrum	Tous les jours, 09h15 (15 min)	Synchronisation, détection des blocages.
Sprint Review	Dernier jour, 14h00–16h00 (2 h)	Démonstration de l'incrément au Product Owner.
Sprint Retrospective	Dernier jour, 16h00–17h00 (1 h)	Amélioration continue du processus.

1.2.3 Definition of Ready (DoR)

Une story ne peut entrer dans un sprint que si :

1. elle a un titre clair et une description ;
2. ses critères d'acceptation sont définis ;
3. ses dépendances sont identifiées ;
4. elle est estimée en points ;
5. le Product Owner a validé sa priorité.

1.2.4 Definition of Done (DoD)

Une story n'est terminée que si :

1. le code est développé et revu (*peer review*) ;
2. les tests unitaires passent (couverture $\geq 70\%$) ;
3. les tests d'intégration passent ;
4. la documentation est mise à jour ;
5. l'incrément est déployé sur l'environnement de staging ;
6. le Product Owner a validé la story.

1.3 Principes DevOps

La démarche DevOps complète Scrum sur l'axe technique :

- **Intégration continue** : chaque *push* déclenche build, tests et analyse qualité (chapitre 5) ;
- **Déploiement continu** : livraison automatique en staging, déploiement en production sur validation manuelle (*approval gate*) ;
- **Infrastructure as Code** : Docker, docker-compose, manifestes Kubernetes optionnels ;
- **Observabilité** : logs centralisés, métriques Prometheus, dashboards Grafana, alerting (chapitre 7) ;
- **Boucle de rétroaction** : les métriques de production alimentent la priorisation du backlog.

Note de cohérence. Les fichiers source annonçaient initialement 284 points planifiés, alors que la somme réelle des stories affectées aux sprints était de **304 points** (écarts sur les sprints 5, 7 et 8). Les fichiers `sprints.json`, `SPRINT-0X.md` et `releases.json` ont été recalés sur cette valeur, et la charge a été rééquilibrée entre sprints (chapitre 3, §3.3) : les trois représentations — product backlog, fichiers opérationnels et présent document — sont désormais alignées.

CHAPITRE 2

Product Backlog

Le backlog est structuré en **10 epics** couvrant l'intégralité du périmètre du cahier des charges, pour un total de **40 user stories** et **304 story points** (suite de Fibonacci : 1, 2, 3, 5, 8, 13).

2.1 Vue d'ensemble des epics

Epic	Intitulé	Priorité	Source CdC	Points
EPIC-001	Gestion des entités métier (CRUD)	Haute	§4.1, §5.2, §7.1	44
EPIC-002	Planification et suivi des visites	Haute	§4.2, §6.1, §6.2	44
EPIC-003	Tableaux de bord et visualisation	Haute	§4.3	42
EPIC-004	Indicateurs et capteurs (préparation)	Moyenne	§4.4	29
EPIC-005	Authentification et sécurité	Haute	§8.2, §8.3, Ann. I	26
EPIC-006	Internationalisation et configuration	Haute	§8.2, §8.3	13
EPIC-007	Service web API tierce	Moyenne	§6.5, §8.2	10
EPIC-008	Fonctionnalités avancées (marché)	Moyenne	§4.6, §4.7	36
EPIC-009	Intelligence artificielle (préparation)	Moyenne	Annexe H	21
EPIC-010	Tests, qualité et documentation	Haute	Chap. 10, 11, Ann. F	39
Total				304

2.2 EPIC-001 — Gestion des entités métier (CRUD)

Opérations CRUD sur fermiers, fermes, sites, ruches, corps/hausses, cadres et agents.

ID	Story	Pts	Priorité	Critères d'acceptation
US-001	CRUD Fermier	5	Haute	CRUD complet avec validation
US-002	CRUD Ferme	5	Haute	Liaison fermier-ferme
US-003	CRUD Site (avec géolocalisation)	8	Haute	Lat/long/altitude + Post-GIS
US-004	CRUD Ruche avec composition	13	Haute	1 corps + 0-5 hausses + 1-10 cadres
US-005	CRUD Agent avec rôles	5	Haute	Rôles : apiculteur, superviseur, responsable, admin
US-006	Contraintes de composition (métier)	8	Haute	Contraintes CHECK SQL + validation Java

2.3 EPIC-002 — Planification et suivi des visites

Planification, approbation, réalisation et rapport de visite.

ID	Story	Pts	Priorité	Critères d'acceptation
US-007	Planifier une visite	8	Haute	Date, raison, actions prévues
US-008	Approuver/refuser un planning	5	Haute	Workflow d'approbation superviseur
US-009	Réaliser une visite et remplir le rapport	13	Haute	Date, heure, durée, constatations, évaluations 1-3
US-010	Ajouter des photos au rapport	5	Haute	Upload multi-photos
US-011	Mode hors-ligne et synchronisation différée	13	Haute	PWA + Service Worker + IndexedDB

2.4 EPIC-003 — Tableaux de bord et visualisation

Calendrier, production, alertes, synthèse ROI.

ID	Story	Pts	Priorité	Critères d'acceptation
US-012	Calendrier matrice agents × ruches	13	Haute	Filtres mois/semaine/saison/année + pop-up
US-013	Tableau de bord production	8	Haute	Seuils paramétrables + collecte/extension
US-014	Tableau de bord alertes sanitaires	8	Haute	Baisse anormale détectée + inspection
US-015	Tableau de bord synthèse et ROI	13	Moyenne	Graphiques production, interventions, ROI

2.5 EPIC-004 — Indicateurs et capteurs (préparation)

Modèle de données ouvert pour l'intégration future des capteurs.

ID	Story	Pts	Priorité	Critères d'acceptation
US-016	Modèle de données Mesure	8	Moyenne	Horodatage, géolocalisation, unités paramétrables
US-017	Interface ingestion REST/MQTT	8	Moyenne	POST <code>/api/mesures</code> + topic MQTT
US-018	Seuils paramétrables et logique d'alerte	8	Moyenne	Anti-rebond / hystérésis
US-019	Conversion d'unités hétérogènes	5	Moyenne	Patron <i>Strategy</i>

2.6 EPIC-005 — Authentification et sécurité

Keycloak (OIDC, fédération Google), RBAC, TLS/X.509.

ID	Story	Pts	Priorité	Critères d'acceptation
US-020	Authentification OAuth 2.0 Google	8	Haute	Flot <i>Authorization Code</i> + JWT
US-021	Authentification locale (repli)	5	Haute	Repli si OAuth indisponible
US-022	Contrôle d'accès RBAC	8	Haute	Matrice 6 profils × 15 fonctions
US-023	Chiffrement TLS 1.3 / X.509	5	Haute	HTTPS forcé + certificats

2.7 EPIC-006 — Internationalisation et configuration

i18n FR/EN/AR, `ConfigZumm.ini`.

ID	Story	Pts	Priorité	Critères d'acceptation
US-024	Internationalisation (FR/EN/AR)	8	Haute	RTL arabe + extensible
US-025	Configuration <code>ConfigZumm.ini</code>	5	Haute	Seuils, unités, modules sans recompilation

2.8 EPIC-007 — Service web API tierce

Exposition d'une API pour applications externes.

ID	Story	Pts	Priorité	Critères d'acceptation
US-026	Service web REST <code>getZummHoneyActualQuantity</code>	5	Moyenne	REST + contrat OpenAPI 3
US-027	Export CSV/TXT	5	Moyenne	Données maîtrisées par l'utilisateur

2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)

Fonctions complémentaires inspirées de HiveTracks, Apiary Book et BeePlus.

ID	Story	Pts	Priorité	Critères d'acceptation
US-028	Photos d'inspection	5	Haute	Attachées au rapport
US-029	Contexte météo local	5	Moyenne	API météo par géolocalisation
US-030	Carte et rayons de butinage	8	Moyenne	MapLibre GL + OpenStreetMap + cercles 1/2/3 km
US-031	Liste de tâches et rappels	5	Haute	Calendrier de rappels
US-032	Suivi de la reine	5	Haute	Historique du statut de la reine
US-033	Récolte et QR code	8	Moyenne	QR par lot + traçabilité

2.10 EPIC-009 — Intelligence artificielle (préparation)

Détection d'anomalie adaptative, pipeline IA.

ID	Story	Pts	Priorité	Critères d'acceptation
US-034	Détection d'anomalie adaptative (EWMA)	13	Moyenne	Ligne de base par ruche + z-score
US-035	Interface microservice IA Python	8	Moyenne	REST/JSON découplé

2.11 EPIC-010 — Tests, qualité et documentation

Plan de tests, UML, rapport, poster.

ID	Story	Pts	Priorité	Critères d'acceptation
US-036	Tests unitaires JUnit 5 + Mockito	8	Haute	70 % de couverture métier
US-037	Tests d'intégration (Testcontainers)	5	Haute	Tests <i>in-container</i>
US-038	Tests de charge k6	5	Moyenne	p95 < 500 ms, erreurs < 1 %
US-039	Diagrammes UML complets	13	Haute	10 diagrammes requis
US-040	Rapport + poster + présentation	8	Haute	Livrables de soutenance

CHAPITRE 3

Planification des sprints

Le projet est découpé en un **sprint de cadrage** (S0) suivi de **8 sprints de construction de 14 jours** (18 semaines au total), du 14 juillet au 16 novembre 2026, avec une capacité de référence de 40 points par sprint. Chaque sprint est décrit ci-dessous avec son *plan d'exécution* : décomposition en tâches techniques, estimation en jours-homme (j-h), livrables et parades aux risques.

Le sprint 0 ne livre aucune fonctionnalité métier et reste hors du calcul de vélocité. Son objet est de lever les quatre décisions structurantes (ADR-001 à ADR-004) et de prouver que la chaîne technique tient de bout en bout *en production* — Spring Boot, PostgreSQL/PostGIS/TimescaleDB, Keycloak, TLS, CI/CD et observabilité assemblés. Sa règle de sortie est stricte : si ce socle n'est pas déployé au terme des deux semaines, le planning est renégocié plutôt qu'absorbé en heures supplémentaires.

Base de chiffrage. Un sprint de 14 jours compte 10 jours ouvrés. Avec une équipe de **trois développeurs à temps plein**, la capacité brute est de $3 \times 10 = 30$ j-h par sprint. Chaque plan d'exécution est chiffré à cette capacité : environ **26,5 j-h** de tâches techniques et **3,5 j-h** de cérémonies Scrum et de revues de code (planning 4 h, dailies, review 2 h, rétrospective 1 h, par personne). Sur les 8 sprints, cela représente **240 j-h** pour 304 points, soit $\approx 0,79$ j-h par story point.

3.1 Calendrier général

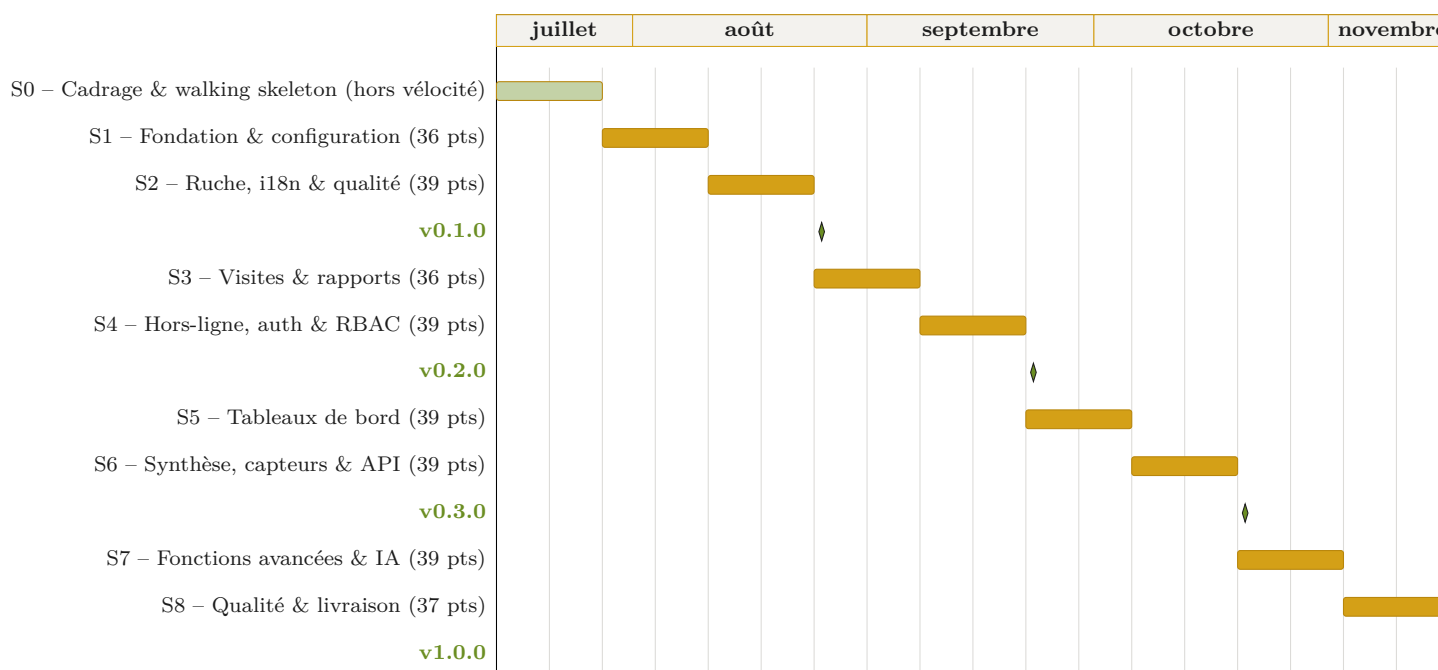


Figure 3.1. Diagramme de Gantt du sprint de cadrage, des 8 sprints de construction et des 4 jalons de release.

3.2 Synthèse des sprints

#	Période	Thème	Points ¹	Stories
0	14/07 → 27/07	Cadrage — décisions & walking skeleton	—	ADR-001 à 004
1	28/07 → 10/08	Fondation — CRUD & configuration	36	US-001/002/003/005/006/025
2	11/08 → 24/08	Ruche, i18n & socle qualité	39	US-004/016/023/024/037
3	25/08 → 07/09	Visites & rapports	36	US-007/008/009/010/028
4	08/09 → 21/09	Hors-ligne, auth. & RBAC	39	US-011/019/020/021/022
5	22/09 → 05/10	Tableaux de bord & pilotage	39	US-012/013/014/027/031
6	06/10 → 19/10	Synthèse, capteurs & API	39	US-015/017/018/026/029
7	20/10 → 02/11	Fonctions avancées & IA	39	US-030/032/033/034/038
8	03/11 → 16/11	Qualité & livraison	37	US-035/036/039/040
Total			304	40 stories

Charge lissée. La répartition initiale plaçait les sprints 5 (50 pts) et 7 (57 pts) très au-dessus de la vélocité cible, tandis que les sprints 2 et 4 restaient à 26 pts : le projet était chargé à l'envers, la surcharge tombant au moment où arrivent aussi les livrables de soutenance. Après rééquilibrage, tous les sprints tiennent dans une fourchette de **36 à 39 points**, sous la capacité de référence de 40, sans décaler aucune date ni retirer aucune story. Le détail de l'arbitrage figure en section 3.3.

3.3 Recommandations d'équilibrage et de pilotage

1. Points recalculés depuis le backlog, après application du rééquilibrage décrit en section 3.3.

3.3.1 Arbitrage retenu

Quatre leviers étaient envisageables : descoper des stories en *Could have* (A), lisser la charge sur les sprints creux (B), scinder les stories lourdes (C), ou réserver un *buffer* de capacité (D).

Le levier **B** a été retenu comme arbitrage principal : il rééquilibre la charge **sans retirer aucune story du périmètre** ni décaler la moindre date — ce qui n'est pas le cas de l'option A, et ce que l'option D ne permet pas à périmètre constant. Les leviers C et D restent disponibles en cours de route, à décider en sprint planning si la vélocité mesurée s'écarte de l'hypothèse (cf. *Pilotage* ci-dessous).

Story	Mouvement et justification	De	Vers
US-025	Configuration <code>ConfigZumm.ini</code> : les seuils doivent être externalisés avant d'être consommés par les écrans.	S2	S1
US-016	Modèle Mesure : simple entité, sans dépendance à l'ingestion — déplacée pour dégager S6.	S6	S2
US-023	TLS 1.3 / X.509 : livrer le chiffrement au dernier sprint est un contresens de sécurité ; HTTPS doit être actif dès les premiers écrans.	S8	S2
US-037	Tests d'intégration Testcontainers : le harnais doit exister <i>avant</i> d'accumuler 40 stories, pas après.	S8	S2
US-028	Photos d'inspection : quasi-doublon technique d'US-010 (photos de rapport), regroupées dans le même sprint.	S7	S3
US-022	RBAC : le workflow d'approbation superviseur (US-008) suppose déjà des rôles ; remontée au plus près de S3.	S5	S4
US-019	Conversion d'unités (<i>Strategy</i>) : story autonome, sert de variable d'ajustement.	S6	S4
US-031	Liste de tâches et rappels : adhère au calendrier d'US-012.	S7	S5
US-027	Export CSV/TXT : suit les tableaux de bord qu'il exporte.	S6	S5
US-015	Synthèse ROI : décalée d'un sprint pour désengorger S5, sans dépendance descendante.	S5	S6
US-029	Contexte météo local : story de priorité moyenne, sert de variable d'ajustement.	S7	S6
US-038	Tests de charge k6 : mesurer la performance <i>avant</i> le sprint final laisse un sprint pour corriger.	S8	S7

Sprint	S1	S2	S3	S4	S5	S6	S7	S8
Avant	31	26	31	26	50	39	57	44
Après	36	39	36	39	39	39	39	37

L'amplitude passe de **26–57 points** à **36–39 points**. Le sprint 7, qui culminait à 142 % de la capacité, revient à 98 %, et plus aucun sprint n'excède la capacité de référence.

Capacité de référence portée à 40 points. La valeur initiale de 38 points était intenable comme plafond : le backlog totalise exactement $8 \times 38 = 304$ points, ce qui imposerait que *chaque* sprint tombe au point près sur 38 — impossible à obtenir avec des estimations Fibonacci (5, 8,

13) sans déformer l'affectation des stories. La capacité de référence est donc fixée à **40 points**, ce qui laisse une marge d'absorption d'environ 5 % et rend la règle vérifiable. Cette valeur reste une hypothèse à recalibrer sur la vitesse mesurée après S2.

3.3.2 Ordre des dépendances

L'ordre de réalisation compte davantage que la charge. Les points suivants ont guidé l'arbitrage ci-dessus :

- **US-022 (RBAC) était trop tardive en S5** : le workflow d'approbation superviseur (US-008, S3) suppose déjà des rôles. Le RBAC minimal reste amorcé en S3 (rôles portés par Agent) et la matrice complète est traitée en S4 ;
- **US-023 (TLS) était en S8** : livrer le chiffrement des échanges au tout dernier sprint expose l'ensemble des environnements intermédiaires. Remontée en S2 ;
- **US-024 (i18n RTL arabe) en S2** : bien placée, mais le rendu RTL doit être testé dès la première maquette — le retrofit RTL est le piège classique ;
- **Spike technique en début de S1** (2-3 jours) sur PostGIS + Spring Boot : c'est le risque identifié du sprint, autant le neutraliser tôt. De même, prototyper Service Worker/IndexedDB pendant S3 pour dérisquer S4.

3.3.3 Pilotage

- **Recalibrer après S2** : la capacité de 40 pts est une hypothèse ; après deux sprints mesurés, replanifier S5-S8 avec la vitesse réelle (c'est là que le choix entre les options A et B se décide) ;
- **Release interne à chaque sprint** (tag `v0.x.y-sprintN`) même si seules les 4 releases publiques comptent : cela teste le pipeline de release avant la `v1.0.0` ;
- **Calendrier** : si le cadrage ne démarre pas réellement le 14/07, décaler toutes les dates est préférable à un sprint 0 amputé — c'est le sprint dont dépendent toutes les décisions suivantes.

3.3.4 Recommandations techniques transversales

- **Migrations de schéma dès S1** (Flyway ou Liquibase) : chaque évolution du modèle est un script versionné — indispensable avec 4 environnements et des contraintes CHECK qui évoluent ;
- **Décisions d'architecture tracées** (ADR — *Architecture Decision Records*) : un fichier court par choix structurant (JSF vs REST+SPA, stockage des photos, stratégie de synchronisation) — réutilisable tel quel dans le rapport de maintenance ;
- **Stockage des photos** : système de fichiers (ou S3/MinIO) + chemin en base, jamais de BLOB en base — avec limite de taille et génération de miniatures dès l'upload ;
- **Revue de code systématique** : chaque PR relue par un pair avant merge sur `develop` (exigence déjà présente dans la DoD, à outiller par une *branch protection rule*) ;
- **Registre des risques** tenu à jour à chaque rétrospective : les risques par sprint listés ci-dessous en sont l'état initial ;
- **UML au fil de l'eau** : commencer les diagrammes dès S2 et les maintenir à chaque sprint plutôt que de tout produire en S8 (US-039, 13 pts, est la plus grosse story du sprint final) ;
- **Gel des fonctionnalités** (*feature freeze*) au jour 10 du sprint 8 : les 4 derniers jours sont réservés aux corrections, aux répétitions de maintenance et à l'empaquetage des livrables.

3.4 Sprint 1 — Fondation : CRUD & configuration

Objectif	Avoir un socle technique fonctionnel avec les entités principales et la configuration externalisée.
Période	28/07/2026 → 10/08/2026 (14 jours) — 36 pts / capacité 40.
Démo	CRUD Fermier/Ferme/Site/Agent + seuils lus depuis <code>ConfigZumm.ini</code> .
Risques	Complexité PostGIS, configuration Spring Boot / Docker.
Burndown idéal	J1 : 36 — J4 : 27 — J7 : 18 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-001	CRUD Fermier	5
US-002	CRUD Ferme	5
US-003	CRUD Site (avec géolocalisation)	8
US-005	CRUD Agent avec rôles	5
US-006	Contraintes de composition (règles métier)	8
US-025	Configuration <code>ConfigZumm.ini</code>	5

3.4.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Spike PostGIS + Spring Boot : datasource, dialecte spatial, requête lat/long de bout en bout	US-003	3	Note technique + config <code>application.yml</code> versionnée
Squelette Maven (module <code>backend/</code>) + <code>frontend/</code> React + CI GitHub Actions (build + tests)	—	3	Dépôt buildable, pipeline vert
Modèle JPA (Fermier, Ferme, Site, Agent) + migrations Flyway	US-001/002/003/005	3,5	Schéma V1 reproductible
Couche DAO/Service + Bean Validation	US-001/002/003/005	3,5	Services testables
Écrans CRUD des 4 entités	US-001/002/003/005	6	4 CRUD démontrables
Contraintes de composition : CHECK SQL + validateurs Java	US-006	3	Règles métier vérifiées aux deux niveaux
Lecteur <code>ConfigZumm.ini</code> (patron <i>Singleton</i>) + rechargement sans redéploiement	US-025	2	Module de configuration
Tests unitaires des services + jeu de données de démonstration	toutes	2,5	Couverture initiale $\geq 50\%$, seed SQL
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : le spike PostGIS occupe les jours 1–2 ; si l'intégration spatiale bloque, solution de repli : colonnes `lat/long` simples en `NUMERIC` et migration PostGIS différée au sprint 5 (les dashboards n'en dépendent pas avant).

Note d'ordonnancement : US-025 remonte de S2. Externaliser les seuils dès le premier sprint évite

de les coder en dur dans les écrans CRUD puis de les extraire ensuite.

3.5 Sprint 2 — Ruche, i18n & socle qualité

Objectif	Modéliser la hiérarchie ruche/corps/hausse/cadre avec contraintes et outiller la qualité dès le début.
Période	11/08/2026 → 24/08/2026 (14 jours) — 39 pts / capacité 40.
Démo	Composition d'une ruche avec règles métier, bascule FR/EN/AR (RTL) et HTTPS actif.
Risques	Contraintes SQL CHECK, patron <i>Composite</i> , rétrofit RTL.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-004	CRUD Ruche avec composition	13
US-024	Internationalisation (FR/EN/AR)	8
US-016	Modèle de données Mesure	8
US-023	Chiffrement TLS 1.3 / X.509	5
US-037	Tests d'intégration (Testcontainers)	5

3.5.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle <i>Composite</i> ruche/corps/hausse/cadre + contraintes (1 corps, 0-5 hausses, 1-10 cadres/élément)	US-004	4	Migration V2 + entités JPA
Interface de composition (vue arborescente, ajout/retrait d'éléments avec validation immédiate)	US-004	4	Écran composition démontrable
i18n FR/EN/AR : externalisation des libellés en ressources de locale, bascule de langue, gabarit RTL	US-024	4	Application trilingue
Test de rendu RTL sur les écrans existants (S1 + composition)	US-024	1,5	Rapport d'écarts RTL
Entité Mesure (horodatage, géolocalisation, unité, source) + migration	US-016	2,5	Modèle extensible capteurs
TLS 1.3 : certificats X.509, HTTPS forcé, redirection 80 → 443 sur staging	US-023	3	Chaîne TLS validée dès S2
Socle de tests d'intégration Testcontainers (PostgreSQL réel, PostGIS + TimescaleDB)	US-037	4	Premier test d'intégration vert en CI
Déploiement continu staging (docker-compose)	—	3,5	Environnement staging alimenté à chaque merge
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : implémenter les règles de composition d'abord en Java (testable rapidement),

puis répliquer en CHECK SQL ; si le patron *Composite* complique le mapping JPA, se limiter à une hiérarchie à trois tables avec clés étrangères et compteurs contraints.

Note d'ordonnement : ce sprint absorbe US-016, US-023 et US-037. Le harnais Testcontainers et la chaîne TLS sont des prérequis d'infrastructure : les placer en S8, comme le faisait la répartition initiale, revenait à valider tardivement des choix qui conditionnent tous les sprints intermédiaires. US-016 n'est qu'une entité et ne dépend pas de l'ingestion (US-017/018, S6).

3.6 Sprint 3 — Visites & rapports

Objectif	Workflow complet : planification → approbation → réalisation → rapport.
Période	25/08/2026 → 07/09/2026 (14 jours) — 36 pts / capacité 40.
Démo	Workflow de visite avec rapport complet et photos d'inspection.
Risques	Volumétrie des photos ; mode hors-ligne (reporté au sprint 4).
Burndown idéal	J1 : 36 — J4 : 27 — J7 : 18 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-007	Planifier une visite	8
US-008	Approuver/refuser un planning	5
US-009	Réaliser une visite et remplir le rapport	13
US-010	Ajouter des photos au rapport	5
US-028	Photos d'inspection	5

3.6.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle Visite/Planning/Rapport + machine à états (planifiée → approuvée → réalisée / refusée)	US-007/008/009	3,5	Migration V3 + diagramme d'états
RBAC minimal : rôles portés par Agent + garde d'accès sur le workflow (anticipation d'US-022, cf. §3.3)	US-008	2,5	Approbation réservée au superviseur
Écrans de planification (date, raison, actions prévues)	US-007	3,5	Formulaire de planification
File d'approbation du superviseur (accepter/refuser + motif)	US-008	2,5	Écran d'approbation
Formulaire de rapport : date, heure, durée, constatations, évaluations 1–3	US-009	4,5	Rapport complet saisissable
Upload multi-photos : stockage fichiers + miniatures + limite de taille	US-010	3,5	Photos attachées au rapport
Réutilisation du module photo pour l'inspection (même pipeline d'upload, cible différente)	US-028	1,5	Photos liées à l'inspection
Spike Service Worker + IndexedDB (préparation S4)	—	2	Prototype de mise en cache
Tests unitaires du workflow (transitions interdites)	US-007/008/009	3	Machine à états couverte
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : le report du mode hors-ligne est déjà acté (S4) ; le spike Service Worker de fin de sprint sécurise cette échéance. Verrouiller la machine à états par des tests avant de brancher l'interface. Stockage des photos sur système de fichiers avec chemin en base et limite de taille — jamais de BLOB (cf. §3.3.4).

Note d'ordonnancement : US-028 remonte de S7. C'est techniquement le même module d'upload qu'US-010 ; les traiter dans le même sprint évite de rouvrir le sujet quatre sprints plus tard pour 1 j-h de travail.

3.7 Sprint 4 — Hors-ligne, authentification & RBAC

Objectif	Permettre la saisie terrain sans réseau et verrouiller les accès par rôle.
Période	08/09/2026 → 21/09/2026 (14 jours) — 39 pts / capacité 40.
Démo	PWA hors-ligne, OAuth Google + repli local, matrice RBAC appliquée.
Risques	Service Workers, IndexedDB, synchronisation différée.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-011	Mode hors-ligne et synchronisation différée	13
US-020	Authentification OAuth 2.0 Google	8
US-021	Authentification locale (repli)	5
US-022	Contrôle d'accès RBAC	8
US-019	Conversion d'unités hétérogènes	5

3.7.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
PWA : manifest + Service Worker (cache de l'app shell)	US-011	3,5	Application installable
File d'attente IndexedDB des saisies hors-ligne (visites, rapports)	US-011	4	Saisie sans réseau fonctionnelle
Synchronisation différée : rejeu à la reconnexion, idempotence, résolution de conflits (<i>last-write-wins</i> + journal)	US-011	4,5	Démo « mode avion »
OAuth 2.0 Google : flot <i>Authorization Code</i> + émission JWT	US-020	3,5	Connexion Google opérationnelle
Authentification locale de repli (hachage BCrypt/Argon2)	US-021	2	Repli fonctionnel hors OAuth
Gestion des secrets dans le pipeline (GitHub Secrets)	US-020	1	Aucun secret en clair dans le dépôt
Généralisation RBAC : matrice 6 profils × 15 fonctions (extension du RBAC minimal de S3)	US-022	3	Matrice appliquée à tous les écrans
Conversion d'unités (patron <i>Strategy</i>)	US-019	2	Conversions couvertes par tests
Tests E2E du parcours hors-ligne (Cypress, réseau coupé)	US-011	3	Scénario automatisé en CI
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : limiter le périmètre hors-ligne à la *saisie de visite/rapport* (pas de consultation complète) ; en cas de blocage sur la résolution de conflits, dégrader en « premier arrivé, premier servi » avec signalement manuel des collisions.

Note d'ordonnancement : US-022 remonte de S5. Le RBAC complet doit suivre immédiatement le RBAC minimal amorcé en S3 et précéder les tableaux de bord, dont la visibilité dépend du profil. US-019 (*Strategy*), story autonome sans dépendance, sert de variable d'ajustement : c'est le fusible du sprint si le hors-ligne dérive.

3.8 Sprint 5 — Tableaux de bord & pilotage

Objectif	Donner à l'apiculteur ses trois vues de pilotage quotidien.
Période	22/09/2026 → 05/10/2026 (14 jours) — 39 pts / capacité 40.
Démo	Calendrier matriciel, tableaux production et alertes, export CSV.
Risques	Performance des agrégations SQL, Chart.js.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-012	Calendrier matrice agents × ruches	13
US-013	Tableau de bord production	8
US-014	Tableau de bord alertes sanitaires	8
US-031	Liste de tâches et rappels	5
US-027	Export CSV/TXT	5

3.8.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Vues SQL agrégées + index (production, visites par période)	US-012/013	4	Requêtes < 200 ms sur données de test
Calendrier matrice agents × ruches : affichage + pop-up de détail	US-012	4,5	Matrice navigable
Filtres mois/semaine/saison/année du calendrier	US-012	2,5	Filtres opérationnels
Dashboard production (Chart.js) + seuils paramétrables via <code>ConfigZümm.ini</code>	US-013	4,5	Graphiques de production
Dashboard alertes sanitaires : règle de baisse anormale (seuil simple, l'EWMA arrive en S7)	US-014	4,5	Liste d'alertes + statut inspection
Liste de tâches et rappels adossée au calendrier	US-031	3	Rappels démontrables
Export CSV/TXT des données maîtrisées par l'utilisateur	US-027	3,5	Export téléchargeable
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : matérialiser les agrégations coûteuses (vues matérialisées rafraîchies à la demande) plutôt que d'optimiser des requêtes à chaud. Si le burndown dérive, scinder US-012 en « affichage matrice » (8 pts) et « filtres/pop-up » (5 pts) — levier C — plutôt que de descoper : les filtres sont la partie la moins critique pour la démo.

Note d'ordonnancement : US-015 (ROI, 13 pts) descend en S6, ce qui ramène ce sprint de 50 à 39 points. US-031 et US-027 remontent car ils s'adossent respectivement au calendrier et aux tableaux qu'ils exportent.

3.9 Sprint 6 — Synthèse, capteurs & API

Objectif	Compléter les tableaux de bord par la synthèse ROI et ouvrir le système aux capteurs et aux tiers.
Période	06/10/2026 → 19/10/2026 (14 jours) — 39 pts / capacité 40.
Démo	Synthèse ROI, ingestion REST/MQTT d'une mesure, appel REST depuis un client externe généré depuis OpenAPI.
Risques	MQTT, idempotence, quota d'appels météo.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-015	Tableau de bord synthèse et ROI	13
US-017	Interface ingestion REST/MQTT	8
US-018	Seuils paramétrables et logique d'alerte	8
US-026	Service web REST <code>getZummHoneyActualQuantity</code>	5
US-029	Contexte météo local	5

3.9.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Synthèse ROI : agrégation production, interventions, coûts + graphiques	US-015	5	Page de synthèse
API REST d'ingestion <code>POST /api/mesures</code> idempotente (clé d'unicité source + horodatage)	US-017	4	Endpoint documenté (OpenAPI)
Broker MQTT (Mosquitto en docker-compose) + consommateur	US-017	4	Ingestion par topic démontrable
Seuils paramétrables + hystérésis/anti-rebond	US-018	3,5	Alertes sans battement
Endpoint REST <code>getZummHoneyActualQuantity</code> + contrat OpenAPI 3	US-026	3,5	Client tiers généré depuis le contrat
API météo par géolocalisation + cache local (quota d'appels)	US-029	3	Contexte météo sur la visite
Tests de contrat des API (REST, validation OpenAPI)	US-017/026	3,5	Suites de contrat en CI
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : si MQTT dérape, livrer d'abord la voie REST (même modèle de données) et brancher MQTT en simple adaptateur ; l'idempotence est testée avant tout branchement de source réelle. US-029 (priorité moyenne, aucune dépendance descendante) est le fusible du sprint.

Note d'ordonnancement : l'entité Mesure (US-016) a été livrée en S2, ce sprint n'a donc plus qu'à brancher l'ingestion dessus. US-015 descend de S5, US-019 remonte en S4.

3.10 Sprint 7 — Fonctions avancées & détection d'anomalie

Objectif	Livrer les fonctions différenciantes et la détection d'anomalie sur les mesures.
Période	20/10/2026 → 02/11/2026 (14 jours) — 39 pts / capacité 40.
Démo	Carte et rayons de butinage, suivi de reine, QR code de lot, alerte EWMA.
Risques	Calibrage de la ligne de base EWMA sans historique réel.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-034	Détection d'anomalie adaptative (EWMA)	13
US-030	Carte et rayons de butinage	8
US-032	Suivi de la reine	5
US-033	Récolte et QR code	8
US-038	Tests de charge k6	5

3.10.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Algorithme EWMA : ligne de base par ruche + z-score, jeu de données simulé pour calibrage	US-034	5,5	Détection validée sur données de test
Intégration EWMA → alertes sanitaires (remplace le seuil simple de S5)	US-034	3	Alerte adaptative en démo
Carte MapLibre GL + OpenStreetMap : sites + cercles de butinage 1/2/3 km	US-030	5	Carte interactive
Suivi de la reine : historique de statut	US-032	3	Chronologie reine
Récolte : lot + QR code + page publique de traçabilité	US-033	5	QR scannable de bout en bout
Tests de charge k6 sur les parcours critiques + corrections de performance	US-038	5	p95 < 500 ms, erreurs < 1 %
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : l'EWMA est calibrée sur un jeu de données simulé — sans historique réel, la ligne de base n'a pas de sens statistique, ce point doit être assumé explicitement en soutenance. Si le calibrage dérape, l'application reste sur le seuil simple de S5 sans autre impact. US-033 est le fusible du sprint (priorité moyenne, aucune dépendance descendante).

Note d'ordonnement : US-038 remonte de S8. Mesurer la performance à l'avant-dernier sprint laisse un sprint entier pour corriger ce qui est trouvé — la placer en S8 revenait à découvrir les problèmes sans avoir le temps de les traiter. US-035 (interface microservice IA) descend en S8 : l'algorithme est ici, son extraction en service découplé l'est là.

3.11 Sprint 8 — Qualité & livraison

Objectif	Atteindre les critères de la DoD sur l'ensemble du produit et préparer les livrables de soutenance.
Période	03/11/2026 → 16/11/2026 (14 jours) — 37 pts / capacité 40.
Démo	Soutenance blanche : démonstration complète, rapport, poster.
Risques	Couverture de tests, concentration documentaire.
Burndown idéal	J1 : 37 — J4 : 28 — J7 : 19 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-035	Interface microservice IA Python	8
US-036	Tests unitaires JUnit 5 + Mockito	8
US-039	Diagrammes UML complets	13
US-040	Rapport + poster + présentation	8

3.11.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Extraction de l'EWMA en microservice Python découplé (API REST/JSON, conteneur docker-compose)	US-035	5	Conteneur IA déployé
Campagne de tests unitaires : combler les trous jusqu'à 70 % de couverture métier	US-036	7	Rapport JaCoCo $\geq 70\%$
Finalisation des 10 diagrammes UML (maintenus depuis S2, cf. §3.3.4)	US-039	6	Dossier UML complet
Rapport, poster, présentation + répétition de soutenance	US-040	8,5	Livrables de soutenance
<i>Feature freeze</i> au jour 10 : corrections et empaquetage uniquement	—	—	v1.0.0 taguée et déployée
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : la couverture de tests ne se rattrape pas en un sprint — d'où le quality gate progressif (chapitre 5) qui doit amener la couverture à $\approx 65\%$ dès la fin de S7 ; les diagrammes UML maintenus au fil de l'eau réduisent US-039 à une passe de mise en cohérence.

Note d'ordonnancement : ce sprint perd US-023 (TLS, → S2), US-037 (Testcontainers, → S2) et US-038 (k6, → S7). Le sprint final ne conserve ainsi que ce qui ne peut pas être anticipé — la campagne de couverture, la mise en cohérence documentaire et la soutenance — au lieu de cumuler infrastructure de sécurité, harnais de test et documentation dans les deux dernières semaines.

3.12 Suivi en cours de sprint

Modalités de suivi. Le *burndown* réel est mis à jour quotidiennement au daily scrum. En fin de sprint, la rétrospective consigne : ce qui a bien fonctionné, ce qui peut être amélioré, et les actions pour le sprint suivant. Les fichiers éditables `SPRINT-0X.md` du dossier `roadmap/operationnel/02_sprints/` servent de support opérationnel ; ce document en est la vue consolidée.

CHAPITRE 4

Gestion des risques et de la charge

Ce chapitre consolide les risques identifiés sprint par sprint (chapitre 3) dans un **registre unique**, coté en probabilité $\times$ impact, et synthétise la charge estimée des plans d'exécution. Le registre est revu à chaque rétrospective (cf. §3.3.4).

4.1 Registre des risques

Cotation : probabilité et impact sur 3 niveaux (1 = faible, 2 = moyen, 3 = fort) ; criticité = probabilité $\times$ impact. Les risques de criticité ≥ 6 appellent une parade *préventive* (engagée avant le sprint concerné), les autres une parade *corrective* (déclenchée si le risque se matérialise).

ID	Risque	Sprint	P	I	C	Parade
R-01	Intégration PostGIS/Spring Boot plus complexe que prévu	S1	2	3	6	Spike jours 1-2 ; repli : lat/long NUMERIC, PostGIS différé en S5.
R-02	Mapping JPA du patron <i>Composite</i> (ruche)	S2	2	2	4	Hiérarchie à trois tables + compteurs contraints si blocage.
R-03	Retrofit RTL arabe coûteux sur écrans existants	S2+	2	2	4	Test RTL dès S2 sur tous les écrans livrés ; gabarit RTL de référence.
R-04	Service Worker / IndexedDB / synchronisation différée	S4	3	3	9	Spike en S3 ; périmètre limité à la saisie ; conflits dégradables en signalement manuel.
R-05	Configuration OAuth (console Google, redirections, secrets)	S4	2	2	4	Repli auth locale (US-021) livré dans le même sprint ; secrets en place avant S4.
R-06	Performance des agrégations SQL des dashboards	S5	2	2	4	Vues matérialisées ; index posés avec les vues ; mesure k6 en S7.

ID	Risque	Sprint	P	I	C	Parade
R-07	Charge de sprint supérieure à la vélocité réelle	tous	2	2	4	Rééquilibrage appliqué (§3.3) : amplitude ramenée de 26–57 à 36–39 pts, sous la capacité de référence de 40. Risque résiduel : cette capacité reste une hypothèse, à recalibrer sur la vélocité mesurée après S2. Fusibles désignés : US-019 (S4), US-029 (S6), US-033 (S7).
R-08	Broker MQTT et idempotence de l'ingestion	S6	2	2	4	Voie REST livrée d'abord ; MQTT en adaptateur ; idempotence testée avant source réelle.
R-09	Calibrage EWMA (S7) et extraction en microservice Python (S8) non aboutis	S7/S8	2	2	4	Découplage par contrat REST ; jeu de données simulé pour le calibrage ; l'application retombe sur le seuil simple de S5.
R-10	Couverture de tests insuffisante en fin de projet	S8	2	3	6	Quality gate progressif (50 % → 70 %) ; couverture ≈ 65 % exigée fin S7.
R-11	Documentation/UML concentrés sur le sprint final	S8	3	2	6	UML au fil de l'eau dès S2 ; ADR réutilisés dans le rapport ; <i>feature freeze</i> J10.
R-12	Dérive de périmètre (fonctions « inspiration marché »)	S7	2	2	4	Priorités MoSCoW gelées au sprint planning ; tout ajout passe par le Product Owner.
R-13	Indisponibilité ponctuelle de l'équipe (période académique)	tous	2	2	4	Capacité planifiée à 90 % (option D) ; stories fusibles par sprint.

Lecture. Trois risques dominant le projet : **R-04** (hors-ligne, criticité 9), **R-07** (surcharge S5/S7) et le couple **R-10/R-11** (qualité et documentation de fin de projet). Les trois sont traités *en amont* : spike S3, options d'équilibrage, quality gate progressif et UML au fil de l'eau.

4.2 Charge estimée par sprint

Synthèse des plans d'exécution du chapitre 3, en jours-homme (j-h) pour 10 jours ouvrés par sprint :

#	Sprint	Points	Charge (j-h)
1	Fondation — CRUD Core	31	14
2	Ruche & composition	26	12,5
3	Visites & rapports	31	14,5
4	Hors-ligne & synchronisation	26	12
5	Tableaux de bord	50	13
6	Capteurs & API	39	11,5
7	Fonctions avancées	57	13
8	Qualité & livraison	44	14
Total		304	104,5

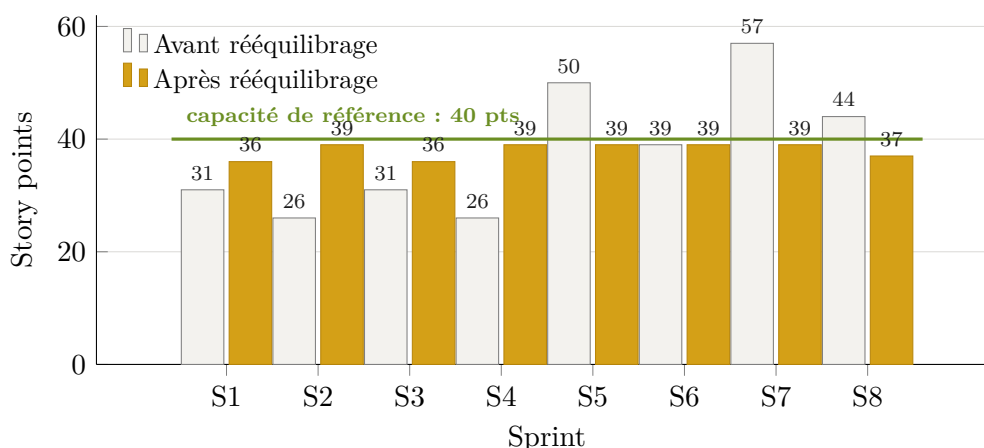


Figure 4.1. Points planifiés par sprint, avant et après rééquilibrage : la charge passe d’une amplitude de 26–57 points à 36–39, sous la capacité de référence (cf. registre R-07).

Dimensionnement de l’équipe. Les plans d’exécution (chapitre 3) sont chiffrés à **30 j-h par sprint**, soit **trois développeurs à temps plein** sur 10 jours ouvrés : environ 26,5 j-h de tâches techniques et 3,5 j-h de cérémonies Scrum et de revues de code, soit **240 j-h** sur les 8 sprints. Le ratio obtenu, $\approx 0,79$ j-h par story point, suppose une équipe déjà à l’aise avec la pile Spring Boot ; pour une équipe qui la découvre, compter plutôt 1 j-h par point — soit ≈ 304 j-h au total, ou ≈ 38 j-h par sprint, que trois personnes ne couvrent pas. C’est l’hypothèse la plus sensible du plan et la première à recalibrer après S1 et S2 (cf. R-07 et *Suivi de la vélocité* ci-dessous). L’hypothèse de réutilisation — module photo, seuils, configuration — doit par ailleurs se vérifier dès S3.

4.3 Suivi de la vélocité

- **Après chaque sprint**, reporter la vélocité réelle (points terminés au sens de la DoD, pas « presque finis ») dans `roadmap/operationnel/02_sprints/sprints.json` ;
- **Après S2**, recalculer la capacité des sprints restants sur la moyenne glissante des deux derniers sprints, et arbitrer les options A/B du §3.3 ;
- **Signal d’alerte** : deux sprints consécutifs sous 80 % de l’engagement déclenchent une re planification des releases (chapitre 6) avec le Product Owner, plutôt qu’une accumulation silencieuse de reste-à-faire.

CHAPITRE 5

Pipeline DevOps (CI/CD)

La chaîne CI/CD s'appuie sur **GitHub Actions** (ou GitLab CI au choix) et se déclenche à chaque *push* ou *pull request*.

5.1 Environnements

Environnement	Branche	URL	Usage
Local	feature/*	http://localhost:8080/zumm	Développement local (Spring Boot)
Dev	develop	https://dev.zumm.local	Intégration continue, tests
Staging	release/*	https://staging.zumm.local	Recette, démo sprint review
Production	main	https://zumm.local	Déploiement manuel validé

5.2 Vue d'ensemble du pipeline

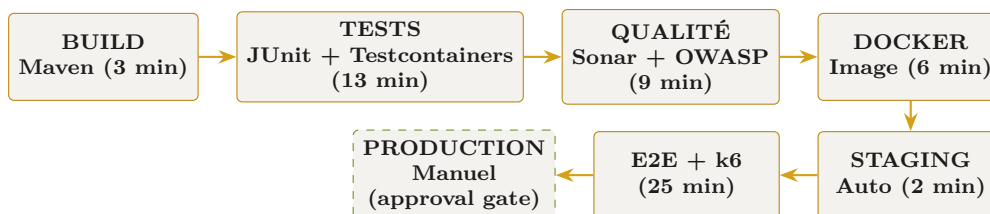


Figure 5.1. Enchaînement des étapes du pipeline CI/CD (durée totale ≈ 1 h).

5.3 Détail des étapes

Étape	Description	Commandes	Seuils / conditions	Durée
Build	Compilation Maven + packaging JAR autoporteur (JDK 17, Maven 3.9+, Spring Boot 3)	<code>mvn clean compile</code> <code>mvn package -DskipTests</code>	—	3 min
Tests unitaires	JUnit 5 + Mockito + JaCoCo	<code>mvn test</code> <code>mvn jacoco:report</code>	couverture $\geq 70\%$, 0 échec	5 min
Tests d'intégration	Testcontainers : PostgreSQL réel (PostGIS + TimescaleDB)	<code>mvn verify -P integration-tests</code>	—	8 min
Analyse qualité	SonarQube + SpotBugs + Checkstyle	<code>mvn sonar:sonar</code> <code>mvn spotbugs:check</code> <code>mvn checkstyle:check</code>	0 bug, 0 vulnérabilité, < 50 <i>code smells</i>	5 min
Sécurité	Scan des dépendances OWASP	<code>mvn dependency-check:check</code>	—	4 min
Build Docker	Image multi-étapes (build Maven puis JRE + JAR Zümm)	<code>docker build -t zumm:\${VERSION} .</code>	—	6 min
Déploiement staging	Déploiement automatique	<code>docker-compose -f docker-compose.staging.yml up -d</code>	branches <code>release/*</code> , <code>main</code>	2 min
Tests E2E	Selenium / Cypress sur staging	<code>mvn test -P e2e</code>	—	10 min
Tests de charge	k6, performance et robustesse	<code>k6 run tests/load/</code>	p95 < 500 ms, erreurs $< 1\%$	15 min
Déploiement production	Déploiement manuel (<i>approval gate</i>)	<code>docker-compose -f docker-compose.prod.yml up -d</code>	branche <code>main</code> + <i>ap-probation</i>	2 min

5.4 Infrastructure as Code

- **Docker** : Dockerfile applicatif multi-étapes (JAR Spring Boot), Dockerfile base de données (PostgreSQL + PostGIS), `docker-compose.yml` pour la stack complète (application + PostgreSQL + Nginx + monitoring) ;
- **Kubernetes** (optionnel, scaling futur) : manifestes `deployment.yml`, `service.yml`, `ingress.yml`, `configmap.yml`.

5.5 Mise en place progressive

Le pipeline complet décrit ci-dessus est la cible ; sa mise en place suit la montée en puissance du projet :

- **CI dès le sprint 1** (build + tests + Sonar), mais CD staging seulement à partir du sprint 2 : inutile de maintenir quatre environnements avant d’avoir une image conteneur stable ;
- **Quality gate progressif** : exiger 70 % de couverture dès le sprint 1 est irréaliste sur du code d’infrastructure — commencer à 50 % et monter de 5 points par sprint jusqu’à la cible ;
- **Gestion des secrets** (GitHub Secrets / vault) intégrée au pipeline avant le sprint 4 : l’authentification OAuth (US-020) en dépend ;
- **Tests E2E et de charge** activés à partir du sprint 5, quand les parcours utilisateur sont stabilisés.

5.6 Stratégie de branches Git

Branche	Rôle
main	Production stable
develop	Intégration continue
release/*	Stabilisation d’une release
feature/*	Développement d’une fonctionnalité
hotfix/*	Correction urgente en production

Fichiers opérationnels. Les fichiers exécutables du pipeline (`github-actions.yml`, `Dockerfile`, `docker-compose.yml`) sont maintenus dans `roadmap/operationnel/03_devops_pipeline/` ; ce chapitre en décrit le contrat (étapes, seuils, conditions).

CHAPITRE 6

Plan de releases

Le versionnage suit **Semantic Versioning** (SemVer : MAJOR.MINOR.PATCH). Quatre releases jalonnent le projet, une à l'issue de chaque paire de sprints.

6.1 Synthèse

Version	Nom	Date prévue	Sprints
v0.1.0	MVP — Fondation	25/08/2026	S1, S2
v0.2.0	Visites & collaboration	22/09/2026	S3, S4
v0.3.0	Analytics & dashboards	20/10/2026	S5, S6
v1.0.0	Release Candidate	16/11/2026	S7, S8

6.2 v0.1.0 — MVP : Fondation

Contenu	CRUD complet des entités métier ; architecture Spring Boot multi-niveaux ; composition de la ruche (corps/hausse/cadre) ; configuration <code>ConfigZumm.ini</code> ; i18n FR/EN/AR ; sécurité TLS 1.3 / X.509 ; harnais de tests d'intégration Testcontainers.
Critère d'acceptation	Application déployable en HTTPS avec données de test.
Statut	Planifiée.

6.3 v0.2.0 — Visites & collaboration

Contenu	Workflow planification/approbation de visite ; rapport de visite complet avec photos ; mode hors-ligne PWA ; authentification OAuth 2.0 et repli local ; contrôle d'accès RBAC complet.
Critère d'acceptation	Parcours utilisateur complet en ligne et hors-ligne.
Statut	Planifiée.

6.4 v0.3.0 — Analytics & dashboards

Contenu	Calendrier agents × ruches ; tableaux de bord production & alertes ; synthèse ROI ; rappels et export CSV/TXT ; ingestion des mesures capteurs ; contexte météo local ; service web API (REST/OpenAPI).
Critère d'acceptation	Dashboards fonctionnels avec données réelles.
Statut	Planifiée.

6.5 v1.0.0 — Release Candidate

Contenu	Fonctionnalités avancées (carte et rayons de butinage, suivi de reine, QR code de traçabilité) ; détection d'anomalie adaptative (IA) ; tests complets (unitaires/intégration/charge) ; documentation & soutenance ; monitoring & alerting.
Critère d'acceptation	Solution complète prête pour la production.
Statut	Planifiée.

Règle de livraison. Une release n'est publiée que si tous les critères de la Definition of Done (§1.2.4) sont satisfaits pour l'ensemble des stories concernées, et si le pipeline CI/CD (chapitre 5) est intégralement vert sur la branche `release/*` correspondante.

CHAPITRE 7

Monitoring & observabilité

L'observabilité repose sur **Prometheus** + **Grafana** (métriques), **ELK** ou **Loki** (logs), **AlertManager** (alertes) et **Jaeger** (tracing distribué).

7.1 Dashboards Grafana

7.1.1 Zümm — Vue d'ensemble (métier)

Widget	Type	Requête PromQL
Ruches actives	stat	count(ruche_status{status='active'})
Visites aujourd'hui	stat	sum(visites_total{date='today'})
Production miel (kg)	graph	sum(production_miel_kg) by (site)
Alertes actives	graph	count(alertes_status{status='active'})
Latence API (p95)	heatmap	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

7.1.2 Zümm — Performance (technique)

Widget	Type	Requête PromQL
CPU JVM	graph	jvm_cpu_usage_percent
Mémoire heap	graph	jvm_memory_heap_used_bytes
Connexions DB	graph	jdbc_connections_active
Requêtes HTTP/s	graph	rate(http_requests_total[1m])

7.1.3 Zümm — Sécurité

Widget	Type	Requête PromQL
Tentatives échouées	stat	sum(auth_failures_total[1h])
Erreurs 4xx/5xx	graph	sum(rate(http_errors_total[5m])) by (status)
Top 10 IP suspectes	table	topk(10, rate(requests_by_ip[5m]))

7.2 Règles d'alerte

Alerte	Condition	Sévérité	Action
Latence API élevée	p95 sur 5 min > 500 ms	Warning	Notifier Slack + escalade si persistant
Taux d'erreurs > 1 %	<code>http_errors</code> / <code>http_requests</code> > 0,01	Critical	PagerDuty + rollback automatique
Connexions DB épuisées	actives / max > 0,9	Critical	Scaler le pool + alerte ops
JVM heap > 85 %	heap utilisé / heap max > 0,85	Warning	Analyse mémoire + redémarrage planifié

7.3 Gestion des logs

- **Niveaux** : ERROR, WARN, INFO, DEBUG ;
- **Corrélation** : en-tête `X-Request-ID` propagé pour le tracing distribué ;
- **Rétention** : 30 jours en stockage chaud, 1 an en stockage froid (S3).

Boucle DevOps. Les métriques métier (production, visites, alertes sanitaires) sont exposées par l'application elle-même : les dashboards de supervision technique et les tableaux de bord fonctionnels du cahier des charges (§4.3) partagent ainsi la même source de vérité.